

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM4522 PROJE RAPORU

Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Veritabanı Güvenliği ve Erişim Kontrolü

Muhammed Eren Köseoğlu

22290604

Github: <https://github.com/Mamyy25/BLM4522>

Proje

Videosu: [https://drive.google.com/file/d/123iYgwJMOH_b7b04qgT2DYQxwCt0sdci/view?usp=drive link](https://drive.google.com/file/d/123iYgwJMOH_b7b04qgT2DYQxwCt0sdci/view?usp=drive_link)

İçindekiler Tablosu

1.Projenin Özeti

2.Veritabanı Yedekleme ve Felaketten Kurtarma Planı

- 2.1 Kapsam Ve Amaç
- 2.2 Kazanımlar Ve Öğretiler
- 2.3 Kod Önizlemesi
 - 2.3.1 Bölüm 1 Veritabanı Kurtarma Modeli
 - 2.3.2 Bölüm 2 Yedekleme Senaryosu
 - 2.3.3 Bölüm 3 Felaket Simülasyonu
 - 2.3.4 Bölüm 4 Kurtarma Senaryosu
 - 2.3.5 Bölüm 5 Otomatik Yedekleme (SQL Server Agent)

3.Veritabanı Erişim Kontrolü

- 3.1 Kapsam Ve Amaç
- 3.2 Kazanımlar Ve Öğretiler
- 3.3 Kod Önizlemesi
 - 3.3.1 Bölüm 1 Veri Şifreleme (Cell Level Encryption)
 - 3.3.2 Bölüm 2 Erişim Yönetimi ve Roller
 - 3.3.3 Bölüm 3 SQL Injection koruması
 - 3.3.4 Bölüm 4 Veritabanı İzleme (Server Auidt)

4.Sonuç

1. Projelerin Genel Özeti

Bu çalışma kapsamında, bir veritabanı yönetim sisteminin (MSSQL) iki yönetsel ayağını oluşturan **Yedekleme & Kurtarma (Backup & Recovery)** ve **Güvenlik & Erişim Kontrolü (Security & Access Control)** konuları üzerine iki farklı proje geliştirilmiştir. Söz konusu projelerin temel amacı; teorik veritabanı yönetim konseptlerini, gerçek hayat senaryolarıyla bir pratik oluşturmaktır.

2. Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Kapsam ve Amaç: Bu proje, olası veri kayıplarına karşı hazırlıklı olmasını sağlamak amacıyla bir yedekleme ve felaket kurtarma senaryosunun tasarlanması ve uygulanmasını kapsar. Sistemde, manuel yedekleme işlemlerinin haricinde; Tam (Full), Fark (Differential) ve İşlem Günlüğü (Transaction Log) yedeklemelerini barındıran otomatik bir görev yöneticisi (SQL Server Agent) eklenmiştir. Ayrıca, bilinçli veri kaybı simülasyonları yapılarak belirli bir zamana dönüş (Point-in-Time Restore) işlemleri test edilmiştir.

Kazanımlar ve Öğretiler:

- **Data Recovery:** Veri tabanının çökmesi veya insan hatası kaynaklı veri silinmesi durumlarında sistemin nasıl geri yükleneceği pratik edilmiştir.
- **Otomasyon:** Yedekleme işlemlerinin manuel olarak yapılması yerine otomatikleştirilmesinin, yönetim yükünü ve hataları azalttığı görülmüştür.
- **Transaction Log:** Özellikle zamana bağlı kurtarma işlemlerinde Transaction Log'ların yönetilmesinin önemi.

Kod Önizlemesi

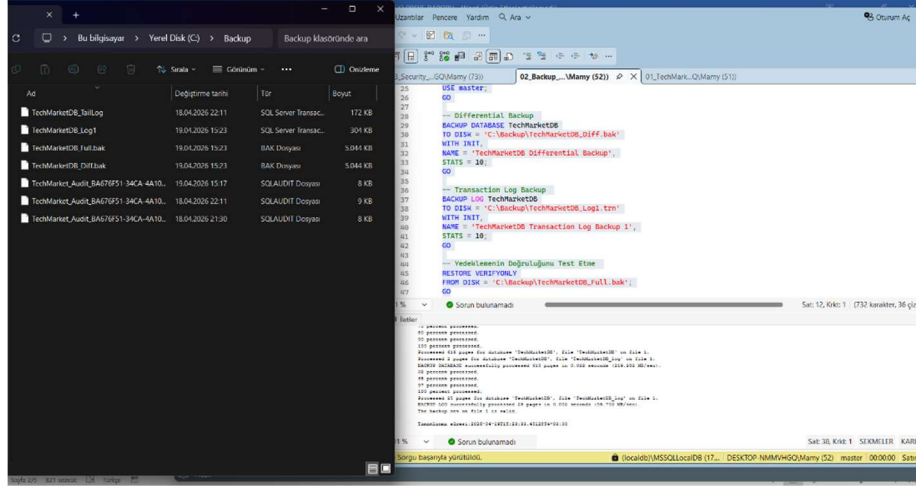
Bölüm 1: Veritabanı Kurtarma Modeli

- **Ne Yapıldı:** Hedefe yönelik "Transaction Log" yedekleri alabilmek için veritabanının modu değiştirildi.
- **Kullanılan Kod:** ALTER DATABASE ... SET RECOVERY FULL;

Bölüm 2: Yedekleme Senaryosu

- **Ne Yapıldı:** Veritabanının tam , sadece değişenlerin yedeği ve son işlemlerin log kayıtları, belirtilen "Backup" dosyasına aktarıldı. Ardından son alınan yedeğin tutarlılığı test edildi.

- **Kullanılan Kod:** BACKUP DATABASE, BACKUP LOG, ve alınan yedeğin fiziksel olarak sağlam olduğunu onaylamak için RESTORE VERIFYONLY komutları.

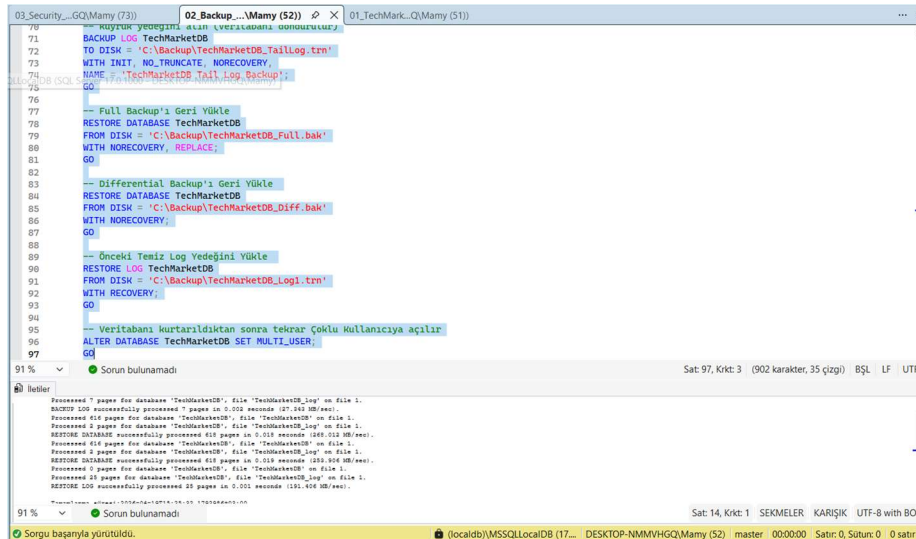


Bölüm 3: Felaket Simülasyonu

- **Ne Yapıldı:** Gerçek bir veri kaybı anı simüle edildi; tabloda bulunan (Siparişler) veriler bilinçli olarak silindi.
- **Kullanılan Kod:** DELETE FROM Siparisler;

Bölüm 4: Kurtarma Senaryosu

- **Ne Yapıldı:** Veriler kaybolduktan sonra aktif işlemlerin Tail Log alınarak sistem donduruldu. Ardından Bölüm 2'de alınan Full, Diff ve Log yedekleri sırasıyla geri yüklendi. Dosyalar sırayla bağlandığı için aralardaki kopuklukları engellemek adına veritabanı ancak son adımda kullanıma geri açıldı.
- **Kullanılan Kod:** Dış müdahaleyi kesmek için SET SINGLE_USER, yedekleri sırasıyla işlemek için RESTORE DATABASE ... WITH NORECOVERY ve geri dönüş için RESTORE LOG ... WITH RECOVERY.



Bölüm 5: Otomatik Yedekleme (SQL Server Agent)

- **Ne Yapıldı:** Sistemin her gece saat 00:00'da insan müdahalesi olmadan Full Backup alması için zamanlanmış bir görev oluşturuldu.
- **Kullanılan Kod:** MSSQL sistem prosedürleri kullanıldı, “sp_add_job” ile görev oluşturuldu, “sp_add_jobstep” ile T-SQL komutu verildi, “sp_add_schedule” ile takvim ayarlandı.

3. Veritabanı Güvenliği ve Erişim Kontrolü

Kapsam ve Amaç: Bu proje, veritabanını korunaklı ve güvenli hale getirmek için yapılan çalışmaları içermektedir. Çalışma kapsamında; kullanıcı kimlik doğrulama mekanizmaları, Rol Tabanlı Erişim Kontrolü ve veri şifreleme yöntemleri bir arada kullanılmıştır. Kullanıcılar (Yönetici, İK, Standart Çalışan vb.) rollere ayrılmış, her role sadece işini yapabilmesi için gereken yetkiler tanımlanmıştır. Ayrıca, tablo bazında hassas veriler (örneğin maaş veya kişisel bilgiler) şifrelenerek yetkisiz erişimlere karşı korumaya alınmıştır.

Kazanımlar ve Öğretiler:

- **Sınırlı Erişim:** Kullanıcıların sadece ihtiyaç duydukları kaynaklara erişebilmesinin güvenlik açısından önemi.
- **Veri Gizliliği** Veri katmanında şifreleme yapılarak hassas verilerin şifrelenmesi.
- **Rol Yönetiminini:** Kullanıcılara tek tek yetki vermek yerine, rollere yetki verip kullanıcıları rollere atamak.
- **Yetki Kısıtlama:** Yetkisiz bir işlem denenmesi durumlarında güvenlik mekanizmaları ile bu isteklerin engellenmesi.

Kod Önizlemesi

Bölüm 1: Veri Şifreleme (Cell-Level Encryption)

- **Ne Yapıldı:** Müşterilerin KrediKartıNo gibi hassas verileri veritabanı seviyesinde açık metin yerine anlamsız (binary) bir yığın olarak şifrelendi. Açık kredi kartı sütunu tamamen silindi.
- **Kullanılan Kod:** Bir anahtar vasıtasıyla veriyi şifrelemek için ENCRYPTBYPASSPHRASE ve şifreli metni çözümleyip geri okumak için DECRYPTBYPASSPHRASE SQL fonksiyonları.
-

```
03_Security...Mamy (72)  X 02_Backup_A...Q(Mamy (52)) 01_TechMark...Q(Mamy (51))
-- PROJE 3: VERİTABANI GÜVENLİĞİ VE ERIŞİM KONTROLÜ
1
2
3 USE Master;
4 GO
5
6 -- 1. VERİ ŞİFRELEME (Hücre Seviyesinde Şifreleme / Cell-Level Encryption)
7
8 USE TechMarketDB;
9 GO
10
11 -- Şifreli verilerin saklanacağı yeni sütun
12 IF NOT EXISTS(SELECT * FROM sys.columns WHERE Name = N'SifreliKrediKarti' AND Object_ID = Object_ID(N'dbo.Musteriler'))
13 BEGIN
14     ALTER TABLE Musteriler ADD SifreliKrediKarti VARBINARY(MAX);
15 END
16 GO
17
18 -- veri şifreleme
19 IF NOT EXISTS(SELECT * FROM sys.columns WHERE Name = N'KrediKartiNo' AND Object_ID = Object_ID(N'dbo.Musteriler'))
20 BEGIN
21     UPDATE Musteriler SET SifreliKrediKarti = ENCRYPTBYPASSPHRASE('CokGizliSifre_123!!!', KrediKartiNo);
22     ALTER TABLE Musteriler DROP COLUMN KrediKartiNo;
23 END
24 GO
25
26 -- Tablodaki verilerin şifreli halini görme
27 SELECT AdSoyad AS Musteri, SifreliKrediKarti AS [Şifrelenmiş Görüntü] FROM Musteriler;
28 GO
29
30 91% 5 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
Sat: 28, Kırk: 3 (790 karakter, 26 çizgi) BŞL LF UTF-8
Sonuçlar Retler
Musteriler Şifrelenmiş Görüntü
1 Ahmet Yılmaz 0x020000007834746A9F81A765CC4CC8C67BEA4FED074856...
2 Ayşe Kaya 0x0200000062B7CA508F81000D24175A2F44674327047A390...
3 Mehmet Öz 0x020000006C72468D8CF6D777E8F0C083F3EF6D243B7FC6...
4 Fatma Çelik 0x020000006A29A847DC088E4B4268CD8562B2F77085C82F...
Sorgu başarıyla yürütüldü. (localdb)\MSSQLLocalDB (17... DESKTOP-NMMVHGQ\Mamy (72) TechMarketDB 00:00:00 Satır: 1, Sütun: 1 4 satır
```

Bölüm 2: Erişim Yönetimi ve Roller

- **Ne Yapıldı:** "Satış" (SalesUser) ve "İnsan Kaynakları" (HRUser) olarak farklı rollere sahip kullanıcılar oluşturuldu. Satış kullanıcılarının şifrelenmiş kredi kartı sütunlarını görmesi engellenirken, İK kullanıcıları sadece "Çalışanlar" tablosunu görebilecek şekilde ayarlandı.
- **Kullanılan Kod:** Oturum ve kullanıcı açmak için CREATE LOGIN / CREATE USER, tablolara spesifik yetki tanımlamak için GRANT SELECT, INSERT, belirli kolonların / tabloların erişimini reddetmek için DENY komutu kullanıldı.

```
03_Security...Mamy (72)  X 02_Backup_A...Q(Mamy (52)) 01_TechMark...Q(Mamy (51))
-- PROJE 3: VERİTABANI GÜVENLİĞİ VE ERIŞİM KONTROLÜ
1
2
3 USE Master;
4 GO
5
6 -- 1. VERİ ŞİFRELEME (Hücre Seviyesinde Şifreleme / Cell-Level Encryption)
7
8 USE TechMarketDB;
9 GO
10
11 -- Şifreli verilerin saklanacağı yeni sütun
12 IF NOT EXISTS(SELECT * FROM sys.columns WHERE Name = N'SifreliKrediKarti' AND Object_ID = Object_ID(N'dbo.Musteriler'))
13 BEGIN
14     ALTER TABLE Musteriler ADD SifreliKrediKarti VARBINARY(MAX);
15 END
16 GO
17
18 -- veri şifreleme
19 IF NOT EXISTS(SELECT * FROM sys.columns WHERE Name = N'KrediKartiNo' AND Object_ID = Object_ID(N'dbo.Musteriler'))
20 BEGIN
21     UPDATE Musteriler SET SifreliKrediKarti = ENCRYPTBYPASSPHRASE('CokGizliSifre_123!!!', KrediKartiNo);
22     ALTER TABLE Musteriler DROP COLUMN KrediKartiNo;
23 END
24 GO
25
26 -- Tablodaki verilerin şifreli halini görme
27 SELECT AdSoyad AS Musteri, SifreliKrediKarti AS [Şifrelenmiş Görüntü] FROM Musteriler;
28 GO
29
30 91% 5 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
Sat: 28, Kırk: 3 (790 karakter, 26 çizgi) BŞL LF UTF-8
Sonuçlar Retler
Musteriler Şifrelenmiş Görüntü
1 Ahmet Yılmaz 0x020000007834746A9F81A765CC4CC8C67BEA4FED074856...
2 Ayşe Kaya 0x0200000062B7CA508F81000D24175A2F44674327047A390...
3 Mehmet Öz 0x020000006C72468D8CF6D777E8F0C083F3EF6D243B7FC6...
4 Fatma Çelik 0x020000006A29A847DC088E4B4268CD8562B2F77085C82F...
Sorgu başarıyla yürütüldü. (localdb)\MSSQLLocalDB (17... DESKTOP-NMMVHGQ\Mamy (72) TechMarketDB 00:00:00 Satır: 1, Sütun: 1 4 satır
```

```
120
121
122
123 USE TechMarketDB;
124 GO
125
126 EXECUTE AS USER = 'SalesUser';
127 GO
128
129 SELECT AdSoyad, SifreLakredikart FROM obo.Musteriler;
130 GO
131
132 SELECT * FROM obo.Calisanlar;
133 GO
134
135 REVERT;
136 GO
137
138
139
```

Sat: 123, Kkt: 1 (162 karakter, 15 çizgi) BŞİ LF UTF-8

Sat: 7, Kkt: 1 SEKMELER KARIŞIK UTF-8 with BOM

Sorgu hataları tamamladı.

Bölüm 3: SQL Injection Koruması

- **Ne Yapıldı:** Dışarıdan veya web ekranlarından gelebilecek potansiyel zararlı SQL komutlarının (örneğin ' OR 1=1 --), veritabanı tarafından çalıştırılabilir bir kod değil, sadece bir **string** olarak algılanması sağlandı.
- **Kullanılan Kod:** Parametrik çalıştırma ortamı sağlayan “sp_executesql stored procedure.

Bölüm 4: Veritabanı İzleme (Server Audit)

- **Ne Yapıldı:** Sistem üzerinde "Kim, Neyi, Ne Zaman" değiştirdi sorusunun yanıtını loglamak üzere bir meknizma “**Audit**” oluşturuldu. Örneğin Calisanlar tablolarındaki "DELETE" işlemleri aktif olarak kayda alındı.
- **Kullanılan Kod:** Sunucu çapında denetim açmak için CREATE SERVER AUDIT, veritabanı seviyesi için CREATE DATABASE AUDIT SPECIFICATION ve olayları belirtmek için ADD (DELETE ON SCHEMA...) yapısı.

4.Sonuç

Verilerin sadece veritabanına kaydedilen satırlardan ibaret olmadığı bu bilgilerin güvenliği, korunması, silinmesi/değiştirilmesi durumunda geri yüklenmesi, kullanıcı yetkileri ile erişimin kısıtlanması gereken hassas içerikler olduğunu ayrıca günümüz dünyasında bazı işlerin otomasyona dökülmesinin önemi kavranmıştır.

